Predicting Registration Error at Lesion Pixels from Fiducial Leverage and Residuals — Simulation Guide

Wei-Che Hung, Department of Systems and Control Engineering, University of Malta · 30 October 2026 · release v1.0-isbi2027

This guide describes the Monte Carlo simulations of the paper (IEEE ISBI 2027, under review): what is simulated, which script produces each figure and table, and how to run them. Equation, figure and table numbers follow the paper.

1. What the simulation tests

Four fiducial markers are registered frame by frame with an affine model, eq. (1). The paper predicts the registration error at a lesion point of interest (POI) from two quantities that every recording provides:

- the **leverage** h_p of the POI, eq. (2), computed once from the marker and POI coordinates in frame-0;
- the **marker residuals** $R^{(k)}$ and their covariance $\hat{\Sigma}^{(k)}$, eq. (3), pooled over the sequence into Σ_{eff}, eq. (4).

The prediction is $\mathrm{RMS}_p = \sqrt{h_p \,\mathrm{tr}(\Sigma_{\text{eff}})}$, eq. (5), extended to the intra-lesion error RMS_L, eq. (6), and to the lesion-boundary area disagreement $\mathbb{E}[D_A]$, eq. (7). The simulation compares each prediction with the Monte Carlo value measured from the same trials.

2. Protocol (paper Sec. 3.4)

Item	Value
Image	640 × 480 pixels
Markers	four; *baseline* configuration, and *expanded* configuration with each marker moved 1.5 times farther from the marker centroid
POIs	65 on a regular grid; ray of 100 POIs from the marker centroid beyond the marker convex hull
Motion per trial	affine: rotation ±5°, translation ±10 pixels, scaling ±2 %
Marker noise	isotropic Gaussian, σ ∈ {0.5, 1, 2, 4} pixels, added to the moved markers
Trials	10 000 per noise level, fixed seed 0
Lesion polygons	triangle, circle, rectangle of equal area at the marker centroid (δ = 0); circles at normalised offsets δ ∈ {0.5, 1.0, 1.5}, σ = 1 pixel

One trial loop serves every artefact: draw the motion, move markers and POIs, add marker noise, fit the affine map back to frame-0, record the POI errors and the marker residuals. The prediction uses only residuals and leverage; the Monte Carlo truth uses the known POI positions.

3. Scripts, outputs and paper objects

Run every command from the repository root. Each script asserts its pinned numbers before drawing.

Script	Produces	Paper object
`src/analysis1/fig1_leverage_map.py`	leverage map, worked frame (seed 1, σ = 1): $h = 1.7806$ at $p_1 = (40, 40)$	Fig. 2
`src/analysis1/fig2_probe_rms.py`	predicted vs simulated RMS for the 65 POIs, both configurations, four noise levels; `results/fig2_probe_rms_stats.json`	Fig. 3, Table 1a
`src/analysis2/fig3_polygon_rms.py`	ray sweep with the 95 % simultaneous band; six lesion polygons with 95 % confidence intervals; `results/fig3_polygon_rms_stats.json`	Fig. 4, Table 1a
`src/analysis3/area_error.py`	predicted and simulated area disagreement D_A for the six polygons; `results/table1_area_error_stats.json`	Table 1b
`src/analysis3/table1_render.py`	`paper/tables/table1_coverage.tex` from the three stats files	Table 1

Supporting modules: `src/common/affine.py` (geometry, motion and noise draws), `src/analysis1/mc_core.py` (trial loop); parity tests: `python -m unittest discover -s src/analysis1`.

4. Run

```
python src/analysis1/fig1_leverage_map.py
python src/analysis1/fig2_probe_rms.py --seed 0 --trials 10000
python src/analysis2/fig3_polygon_rms.py --seed 0 --trials 10000 --sigma 1
python src/analysis3/area_error.py --seed 0 --trials 10000 --sigma 1
python src/analysis3/table1_render.py
```

Requirements: Python 3.10 or later, NumPy, SciPy, Matplotlib. Figures go to `paper/figures/`, statistics to `results/`; every number in Table 1 is read from those files at render time.

Code availability and contact. This repository holds the guide; the scripts are released here after the review round. Wei-Che Hung, wei-che.hung@um.edu.mt or wolfmib@gmail.com.